

Implementasi Sistem Client–Proxy–Server dengan Caching, Multithreading, dan Pengukuran QoS Berbasis UDP



Disusun Oleh:

103012400371 A Muh Imran Ramadhan A

103012400230 Ahmad Kadhim

103012400386 Hamad Dafala

Dosen Pengampu:

TAUFAN ZANDY ANDRIAN, S.T.,M.T.

JARINGAN KOMPUTER

FAKULTAS INFORMATIKA

PROGRAM STUDI S1 INFORMATIKA

BAB I.....	3
Pendahuluan.....	3
BAB II.....	4
Pembagian Tugas.....	4
BAB III.....	5
Implementasi Sistem.....	5
1. Arsitektur dan Alur Komunikasi.....	5
a. Alur HTTP Web Proxy (TCP).....	5
b. Alur Pengukuran Kualitas Jaringan (UDP QoS).....	5
2. Dokumentasi Kode Inti.....	6
a. Web Server.....	6
b. Proxy.....	7
c. Client.....	8
3. Justifikasi Desain.....	10
BAB IV.....	11
Analisis QoS dan Multithreading.....	11
1. Perhitungan QoS.....	11
2. Analisis Faktor yang Mempengaruhi Performa.....	11
3. Studi Komparatif: Cache HIT vs Cache MISS.....	12
4. Evaluasi Multithreading: Skalabilitas dan Efektivitas.....	12
BAB V.....	14
Troubleshooting (Penanganan Kendala).....	14
BAB VI.....	16
Kesimpulan dan Saran.....	16
1. Rangkuman Capaian.....	16
2. Rekomendasi Pengembangan.....	16
REFERENSI.....	17

BAB I

Pendahuluan

Di era digital seperti sekarang, hampir semua aktivitas kita melibatkan komunikasi jaringan mulai dari membuka halaman web, streaming, hingga transfer data. Di balik semua itu, ada mekanisme teknis yang bekerja secara diam-diam: protokol komunikasi, server yang melayani permintaan, hingga sistem perantara yang mengoptimalkan performa. Sayangnya, banyak dari kita hanya menjadi pengguna tanpa benar-benar memahami cara kerja di baliknya.

Tugas Besar ini hadir sebagai upaya untuk memahami hal tersebut secara langsung. Kami membangun sebuah sistem jaringan sederhana namun lengkap dari awal menggunakan Python, yang terdiri dari tiga komponen utama yaitu: web server, proxy server yang berfungsi sebagai perantara sekaligus tempat menyimpan cache, serta client yang dapat mengakses konten maupun mengukur kualitas jaringan melalui protokol UDP.

Seluruh implementasi dilakukan menggunakan modul bawaan Python (socket, threading, os) tanpa library jaringan tambahan. Pendekatan ini memang sedikit lebih menantang dan baru untuk kami, tapi kami jadi bisa melihat langsung bagaimana HTTP request dibentuk, bagaimana koneksi TCP dikelola, dan bagaimana paket UDP digunakan untuk mengukur performa jaringan secara nyata.

Laporan ini menyajikan dokumentasi lengkap dari sistem kami bangun, mulai dari arsitektur dan penjelasan kode, hasil analisis Quality of Service (QoS), kendala yang kami hadapi selama pengembangan, hingga saran pengembangan ke depan. Semoga laporan ini tidak hanya menjadi dokumentasi tugas, tetapi juga menjadi gambaran nyata dari proses belajar yang kami jalani.

BAB II

Pembagian Tugas

Berikut adalah deskripsi kontribusi masing-masing anggota kelompok dalam implementasi dan pengujian sistem jaringan.

Nama	Komponen	Deskripsi
Ahmad Kadhim (103012400230)	Client (client.py)	
A MUH Imran Ramadhan A (103012400371)	Proxy Server (proxy.py)	Proxy + server = Proxy Server
Hamad Dafala (103012400386)	Web Server (webserver.py)	Web web server server = web server

BAB III

Implementasi Sistem

1. Arsitektur dan Alur Komunikasi

Sistem yang diimplementasikan pada tugas besar ini mengadopsi arsitektur Client-Server berbasis Socket Programming tanpa menggunakan framework eksternal. Sistem ini terdiri dari tiga komponen utama yang berkomunikasi secara aktif menggunakan protokol TCP serta UDP.

Secara umum, alur komunikasi antar komponen dibagi menjadi dua berdasarkan protokol transport yang digunakan:

a. Alur HTTP Web Proxy (TCP)

- Permintaan Klien (*Client Request*): client.py menginisiasi koneksi TCP ke proxy.py pada port 8080 dan mengirimkan berkas HTTP GET *request* (misalnya meminta berkas /index.html).
- Pemeriksaan Cache (*Cache Checking*): proxy.py menerima *request* tersebut dan memeriksa direktori penyimpanan lokal (cache/).
 - *Cache HIT*: Jika salinan berkas yang diminta sudah ada di dalam direktori cache/, proxy akan langsung menyajikan berkas tersebut kembali ke client tanpa melakukan kontak ke web server.
 - *Cache MISS*: Jika berkas belum pernah disimpan, proxy akan meneruskan (*forward*) *request* tersebut ke webserver.py melalui koneksi TCP port 8000.
- Respon Server & Penyimpanan (*Server Response & Storage*): webserver.py membaca berkas statis dari folder HTML/, lalu mengirimkannya kembali ke proxy. Jika status respon bernilai 200 OK, proxy akan menyimpan berkas tersebut ke dalam folder cache/ sebelum meneruskannya sampai ke client.

b. Alur Pengukuran Kualitas Jaringan (UDP QoS)

- Untuk pengujian performa jaringan (QoS), client.py akan beralih fungsi menjadi alat ukur (*pinger*) yang mengirimkan sejumlah paket data UDP secara langsung (*direct*) ke webserver.py pada port 9000 tanpa melalui proxy.
- webserver.py bertindak sebagai *QoS Echo Server* yang memantulkan kembali (*reflect*) persis setiap *payload* paket UDP yang diterimanya.
- client.py kemudian menghitung selisih waktu kirim dan terima paket untuk mengkalkulasi statistik performa berupa nilai *Round-Trip Time* (RTT min/avg/max), *Packet Loss*, *Jitter*, serta *Throughput* jaringan.

2. Dokumentasi Kode Inti

a. Web Server

Web server minimalis yang berjalan di atas raw socket Python tanpa framework eksternal apapun. Server ini menjalankan dua layanan secara bersamaan menggunakan threading,

```
# UDP server di thread terpisah
udp_thread = threading.Thread(target=start_udp_server, daemon=True)
udp_thread.start()

# TCP server di main thread (blocking)
try:
    start_tcp_server()
```

Gambar 1: Dua layanan via Threading

```
# -----
#  KONFIGURASI
# -----
TCP_HOST = "0.0.0.0"
TCP_PORT = 8000
UDP_HOST = "0.0.0.0"
UDP_PORT = 9000

# Direktori tempat file HTML disimpan
BASE_DIR = "HTML"
```

Gambar 2: Config TCP/UDP dengan Directory HTML

TCP di port 8000 untuk melayani permintaan HTTP GET terhadap file statis dari direktori HTML/, dan UDP di port 9000 sebagai echo server untuk keperluan pengukuran kualitas jaringan (QoS). Setiap koneksi TCP ditangani oleh thread tersendiri sehingga server dapat melayani banyak klien secara konkuren, dengan perlindungan dasar terhadap path traversal untuk mencegah akses file di luar direktori yang diizinkan

```
if safe_path.startswith("../"):
    body = b"<h1>403 Forbidden</h1>"
    conn.sendall(build_response(403, "Forbidden", body))
    log("TCP", f"{client_ip} GET {path} - 403 Forbidden (path traversal)")
    return
```

Gambar 3: Perlindungan Path Traversal

```

while True:
    try:
        conn, addr = server.accept()
        t = threading.Thread(target=handle_tcp_client, args=(conn, addr), daemon=True)
        t.start()

```

Gambar 4: One Thread per connection

b. Proxy

Bertindak sebagai perantara (reverse proxy) antara klien dan web server, dengan kemampuan caching berbasis file sistem untuk mempercepat respons pada permintaan berulang. Ketika sebuah permintaan GET masuk, proxy memeriksa apakah respons untuk path tersebut sudah tersimpan di direktori cache/, jika ada (cache hit), data langsung dikirim ke klien tanpa meneruskan permintaan ke web server. Jika tidak (cache miss), proxy mem-forward permintaan ke web server, menyimpan respons ke cache hanya jika status code adalah 200, lalu mengirimkan hasilnya ke klien.

```

def get_from_cache(path):
    """Return bytes isi cache jika ada, None jika tidak."""
    cache_file = path_to_cache_filename(path)
    with cache_lock:
        if os.path.exists(cache_file):
            with open(cache_file, "rb") as f:
                return f.read()
    return None

```

Gambar 1: Cache hit, kirim tanpa forward

Konkurensi dikelola dengan satu thread per koneksi klien, dengan tambahan `threading.Lock` (`cache_lock`) untuk memastikan operasi baca/tulis ke file cache tidak menimbulkan race condition. Proxy juga menangani kasus error secara eksplisit: timeout menghasilkan 504, kegagalan koneksi menghasilkan 502, dan request non-GET langsung ditolak dengan 405.

```

# -----
# HELPER: CEK CACHE
# -----
def get_from_cache(path):
    """Return bytes isi cache jika ada, None
    cache_file = path_to_cache_filename(path)
    with cache_lock:
        if os.path.exists(cache_file):
            with open(cache_file, "rb") as f:
                return f.read()
    return None

# -----
# HELPER: SIMPAN KE CACHE
# -----
def save_to_cache(path, data):
    """Simpan response bytes ke file cache."""
    cache_file = path_to_cache_filename(path)
    with cache_lock:
        with open(cache_file, "wb") as f:
            f.write(data)

```

Gambar 2: Cache Lock

c. Client

Alat uji dua-mode yang dapat beroperasi sebagai HTTP client (mode TCP) maupun QoS measurement tool (mode UDP), dipilih melalui argumen command line `-mode tcp` atau `-mode udp`. Pada mode TCP, client membuka koneksi langsung ke proxy di port 8080, mengirimkan HTTP GET request, mengukur RTT dari waktu kirim hingga seluruh respons diterima, lalu menampilkan status line, ukuran respons, header, dan 500 karakter pertama body.


```

t_send = time.time()
s.sendall(request.encode("utf-8"))

response = b""
while True:
    try:
        chunk = s.recv(4096)
        if not chunk:
            break
        response += chunk
    except socket.timeout:
        break

t_recv = time.time()
s.close()

rtt_ms = (t_recv - t_send) * 1000

```

Gambar 1: Pengukuran RTT pada mode TCP

Pada mode UDP, client mengirimkan sejumlah paket ke UDP echo server, mengukur RTT per paket, lalu menghitung statistik agregat: packet loss, RTT min/avg/max, jitter (standar deviasi selisih RTT antar-paket), dan throughput dalam kbps.

```

for seq in range(1, n + 1):
    timestamp = time.time()
    payload = f"Ping {seq} {timestamp}".encode("utf-8")
    total_payload += len(payload)

    try:
        s.sendto(payload, (host, UDP_PORT))
        t_send = time.time()

        data, _ = s.recvfrom(1024)
        t_recv = time.time()

        rtt_ms = (t_recv - t_send) * 1000
        rtt_list.append(rtt_ms)

        echo_ok = (data == payload)
        status = "OK" if echo_ok else "ECHO MISMATCH"

```

Gambar 2: RTT per paket mode UDP

3. Justifikasi Desain

Pemisahan tanggung jawab (separation of concerns) menjadi prinsip utama yang support seluruh project ini. Web server, proxy, dan client dipisahkan menjadi tiga proses independent yang berkomunikasi hanya melalui socket ini memungkinkan setiap komponen dikembangkan, diuji, dan dijalankan secara terpisah, bahkan di mesin yang berbeda jika diperlukan. Pilihan menggunakan raw socket Python tanpa library HTTP (seperti `http.server` atau `requests`) adalah keputusan untuk tujuan edukasi dan kontrol penuh: setiap byte header HTTP dibentuk secara eksplisit, sehingga behavior sistem sepenuhnya transparan dan mudah dilacak saat debugging.

Threading model one-thread-per-connection dipilih karena kesederhanaannya. Mudah dipahami dan cukup memadai untuk beban uji skala kecil hingga menengah. Kelemahan model ini pada beban sangat tinggi (thread overhead besar) disadari dan dimitigasi sebagian dengan `server.listen(50)` (backlog queue 50 koneksi).

Penggunaan `threading.Lock` pada `proxy.py` untuk cache adalah contoh penerapan thread safety yang minimal namun tepat sasaran: hanya operasi file yang di kritik, bukan seluruh logic proxy, sehingga konkurensi tidak terganggu lebih dari yang perlu. Caching berbasis file sistem (bukan in-memory seperti dict) dipilih agar cache persisten melewati restart proxy dan mudah diinspeksi secara manual, trade-off yang masuk akal untuk konteks ini mengingat web server melayani file statis yang tidak berubah.

Desain UDP echo server yang semiminal mungkin (pantul kembali payload tanpa modifikasi) memastikan pengukuran RTT di sisi client mencerminkan latensi jaringan murni tanpa bias dari pemrosesan server, yang merupakan standar dalam tool QoS seperti ping.

BAB IV

Analisis QoS dan Multithreading

1. Perhitungan QoS

Pengujian QoS dilakukan menggunakan client mode UDP dengan mengirimkan 20 paket ping ke UDP echo server. Berikut adalah contoh hasil pengukuran pada kondisi jaringan loopback (localhost):

Mertik QoS	Nilai
RTT Minimum	0.067 ms
RTT Average	0.088 ms
RTT Maximum	0.136 ms
Packet Loss	0.0 %
Jitter	0.016 ms
Throughput	1.988 kbps

2. Analisis Faktor yang Mempengaruhi Performa

Beberapa faktor yang dapat mempengaruhi kualitas performa jaringan dalam sistem ini antara lain:

- Beban Server (load): Semakin banyak thread aktif secara bersamaan, semakin tinggi overhead context switching OS, yang dapat meningkatkan RTT dan jitter.
- Ukuran payload UDP: Payload yang lebih besar meningkatkan total data yang ditransmisikan namun juga meningkatkan kemungkinan fragmentasi pada jaringan non-lokal
- Timeout konfigurasi: Nilai `UDP_TIMEOUT = 1.0` detik menentukan batas waktu tunggu balasan. Nilai terlalu kecil dapat menyebabkan false packet loss pada jaringan dengan latensi tinggi.

3. Studi Komparatif: Cache HIT vs Cache MISS

Salah satu manfaat utama proxy caching adalah pengurangan latensi pada akses berulang. Perbandingan performa antara kondisi cache HIT dan MISS dapat dilihat pada tabel berikut. Hasil di atas menunjukkan

Skenario	Rata-rata Latensi	Keterangan
Cache MISS (request pertama)	5.4 ms (sisi Proxy)	Proxy harus meneruskan request ke web server port 8000 karena cache belum tersedia di local disk, menghasilkan durasi pemrosesan sebesar 5.4 ms
Cache HIT (request ulang)	< 1 ms	Proxy langsung membalas request menggunakan file biner yang telah tersimpan di direktori cache lokal tanpa perlu menghubungi web server kembali.
Cache HIT	< 2 ms	Beberapa client mengakses file biner index.html yang sama secara konkuren.

4. Evaluasi Multithreading: Skalabilitas dan Efektivitas

Model konkurensi pada aplikasi Web Server dan Proxy Server diimplementasikan menggunakan strategi Thread-pre-Connection via fungsi `threading.Thread()`. Berdasarkan hasil pengujian, efektivitas dan skalabilitas model ini dapat dievaluasi sebagai berikut:

Efektivitas Penanganan Koneksi Simultan

Ketika Client melakukan request HTTP TCP ke port 8080, Proxy Server mendeteksi koneksi dan langsung memicu pemisahan jalur eksekusi baru. Secara bersamaan, Web Server pada port 8000 juga mencatat adanya lonjakan *activate thread* menjadi 2 ketika menangani request operan dari proxy. Hal ini membuktikan bahwa arsitektur *multithreading* yang dirancang berhasil mengisolasi setiap request ke dalam unit eksekusi independen. Jika terdapat client lain yang masuk disaat bersamaan, client tersebut tidak perlu mengantri (*non-blocking*) karena akan langsung dialokasikan ke thread baru yang terpisah.

Dampak terhadap Skalabilitas Sistem

Kelebihan: Sederhana dalam implementasi kode dan sangat responsif untuk beban kerja rendah hingga menengah. Pemisahan tugas antara thread HTTP (TCP) dan thread QoS Echo Server (UDP) pada web server terbukti berjalan sangat stabil, ditunjukkan dengan nilai rata-rata RTT UDP yang sangat konsisten di angka 0.888 ms tanpa terganggu oleh adanya aktivitas komunikasi TCP sebelumnya.

Keterbatasan: Model Thread-per-Connection ini memiliki batas skalabilitas (scalability bottleneck). Karena setiap thread di Python membutuhkan alokasi memori stack space tersendiri di tingkat sistem operasi, lonjakan ribuan request dari client secara serentak akan memicu pembengkakan penggunaan memori RAM (Overhead Memory). Selain itu, adanya Global Interpreter lock (GIL) pada Python membuat thread-thread ini bergantian menggunakan inti CPU, sehingga tidak efektif untuk menangani skalabilitas ekstrim berskala industri.

BAB V

Troubleshooting (Penanganan Kendala)

Selama proses implementasi dan pengujian sistem, terdapat beberapa kendala teknis yang ditemukan. Berikut adalah dokumentasi kendala serta analisis akar masalah dan solusi yang diterapkan.

Masalah	Penyebab	Solusi
Kehilangan Data Transaksi Aktivitas	Pada rancangan awal sistem, fungsi log() hanya menggunakan perintah print(). Akibatnya, riwayat <i>request</i> , status <i>caching</i> , dan hasil tabel QoS langsung hilang dari memori begitu jendela CMD ditutup.	Memperbarui fungsi log() di setiap berkas dengan menambahkan mekanisme <i>file handling</i> menggunakan <i>mode append</i> ("a") agar setiap riwayat teks otomatis tertulis secara permanen ke file .txt.
Direktori Utama Berantakan	Setelah file log berhasil tersimpan, file teks (.txt) justru terbuat dan menumpuk di folder <i>root</i> proyek, sehingga membingungkan pengguna karena bercampur dengan file kode utama .py.	Mengintegrasikan fungsi <code>os.makedirs("logs", exist_ok=True)</code> dan <code>os.path.join("logs", ...)</code> di dalam kode agar sistem otomatis mengisolasi seluruh berkas log ke dalam satu folder khusus bernama logs/.
Error 'NameError: name 'os' is not defined'	Library os dipanggil di dalam fungsi pencatatan log otomatis (<code>os.makedirs()</code>), namun modulnya belum di-import di bagian atas berkas client.py	Menambahkan baris instruksi <code>import os</code> secara eksplisit pada baris awal kompilasi berkas client.py sebelum fungsi logging dieksekusi.
Korupsi Data Cache (<i>Race Condition</i>)	Beberapa <i>worker thread</i> di dalam proxy.py mencoba menulis (write) atau membaca (read) file cache statis yang sama di direktori cache/ secara simultan.	Mengimplementasikan mekanisme <i>file locking</i> menggunakan library bawaan <code>threading.Lock()</code> yang membungkus fungsi <code>get_from_cache</code> dan <code>save_to_cache</code> .
Akses File Terlarang (<i>Path Traversal</i>)	Potensi kerentanan keamanan di mana klien mengirimkan request manipulasi string path	Mengintegrasikan filter validasi menggunakan fungsi <code>os.path.normpath(path.lstrip("/</code>

	seperti GET ../../etc/passwd untuk menembus keluar folder HTML/.	/")) di webserver.py. Jika string path terdeteksi menyeberang ke direktori atas (..), server langsung melempar respon 403 Forbidden.
--	--	--

BAB VI

Kesimpulan dan Saran

1. Rangkuman Capaian

Kelompok berhasil membangun web server berbasis raw socket yang melayani file statis melalui protokol HTTP/1.1 (TCP port 8000) sekaligus menjalankan UDP echo server (port 9090) untuk keperluan pengukuran QoS secara bersamaan dalam satu proses menggunakan threading. Implementasi ini dilakukan sepenuhnya tanpa framework eksternal, sehingga memberikan pemahaman mendalam tentang cara kerja protokol jaringan di tingkat socket.

Proxy server berhasil diimplementasikan dengan mekanisme caching berbasis file sistem yang mampu membedakan kondisi Cache HIT dan Cache MISS secara transparan melalui log. Penggunaan `threading.Lock()` membuktikan penerapan thread safety yang tepat sasaran untuk mencegah race condition pada operasi baca/tulis cache yang diakses oleh banyak thread secara konkuren.

Client berhasil mengimplementasikan dua mode operasi: mode TCP untuk pengiriman HTTP request melalui proxy, dan mode UDP untuk pengukuran metrik QoS secara kuantitatif meliputi RTT minimum/rata-rata/maksimum, packet loss, jitter, dan throughput.

Sistem terbukti dapat beroperasi dalam dua skenario topologi: single-device menggunakan loopback (127.0.0.1) dan multi-device menggunakan jaringan Wi-Fi bersama (LAN IP), dengan penyesuaian konfigurasi yang minimal hanya pada variabel IP di file `proxy.py` dan `client.py`.

2. Rekomendasi Pengembangan

Untuk fitur caching, implementasi saat ini tidak memiliki mekanisme invalidasi atau expiry cache. Pengembangan selanjutnya dapat menambahkan TTL (Time-to-Live) pada setiap entri cache dan mendukung header HTTP standar seperti Cache-Control dan ETag, sehingga cache dapat diperbarui secara otomatis ketika konten di web server berubah tanpa perlu menghapus file cache secara manual.

REFERENSI

- [1] Python Software Foundation. (2024). socket — Low-level networking interface. Python 3 Documentation. <https://docs.python.org/3/library/socket.html>
- [2] Python Software Foundation. (2024). threading — Thread-based parallelism. Python 3 Documentation. <https://docs.python.org/3/library/threading.html>
- [3] Fielding, R., et al. (1999). RFC 2616: Hypertext Transfer Protocol — HTTP/1.1. Internet Engineering Task Force (IETF). <https://tools.ietf.org/html/rfc2616>
- [4] Postel, J. (1980). RFC 768: User Datagram Protocol. Internet Engineering Task Force (IETF). <https://tools.ietf.org/html/rfc768>
- [5] Stallings, W. (2021). Data and Computer Communications (10th ed.). Pearson Education.
- [6] Kurose, J. F., & Ross, K. W. (2022). Computer Networking: A Top-Down Approach (8th ed.). Pearson Education.
- [7] Python Software Foundation. (2024). os.path — Common pathname manipulations. Python 3 Documentation. <https://docs.python.org/3/library/os.path.html>
- [8] MDN Web Docs. (2024). HTTP response status codes. Mozilla Developer Network. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>